

编译原理 2025 风格模拟卷04（题目与考试标准答案）

卷面说明

本卷按 Slides/Slides/handout 与三份 Recap PPT 更新，主线覆盖前端自动机与 LL、中端 IR/SSA、数据流、符号执行、后端目标代码、寄存器分配、指令调度以及过程间/指针分析。总分 100 分。

A 卷题目

一、词法分析（15 分）

正则表达式：

```
a (b|c)* c
```

1. 使用 Thompson 构造法构造等价 NFA。
2. 使用子集构造法构造 DFA，写明 DFA 状态含义。
3. 最小化 DFA。

二、LL 语法分析（15 分）

文法：

```
M -> N O
N -> N comma | name
O -> semi | end
```

1. 消除左递归。
2. 写 FIRST 和 FOLLOW。
3. 写预测分析表。
4. 判断消除左递归后的文法是否为 LL(1)。

三、指令调度（15 分）

给定指令：

```
I1: R1 = R2 + R3
I2: R4 = R1 + R5
I3: R2 = R6 + R7
I4: R1 = R8 + R9
I5: R10 = R4 + R1
```

1. 写出指令调度的目的。
2. 写出 RAW、WAR、WAW 的定义。
3. 构建依赖图，WAR/WAW 用虚线表示。
4. 在允许寄存器重命名的条件下给出一个合法重排序列。

四、支配问题（15 分）

CFG：循环带出口分支

```
Entry->H
H->Body
Body->If
If->Body
If->Exit
```

1. 写出 Dom、PostDom、Dom Frontier 的定义。
2. 计算每个节点的 Dom 集和 idom。
3. 写出支配边界。
4. 说明 SSA 中 phi 函数应放在什么位置。

五、符号执行与 SAT（20 分）

公式：

```
F = ((a4 and b4) or c4)
```

1. 写出 Path-sensitive、Flow-sensitive、Context-sensitive 的定义。
2. 写出 CNF 的定义。
3. 使用 Tseitin 算法把 F 转成 CNF。
4. 证明 2025 卷中的两组公式同时可满足。

六、Andersen 指针分析（20 分）

程序：

```
a = &O1  
b = &O2  
c = a  
d = b  
*c = d  
e = *a
```

1. 对每条语句写 Andersen 约束。
2. 根据约束构建闭包。
3. 写出最终 points-to 集合。

七、PPT 新增重点：寄存器分配（10 分）

活跃集合如下：

```
p1: {a,d}  
p2: {a,b,d}  
p3: {b,d}  
p4: {b,c,d}  
p5: {c,d}
```

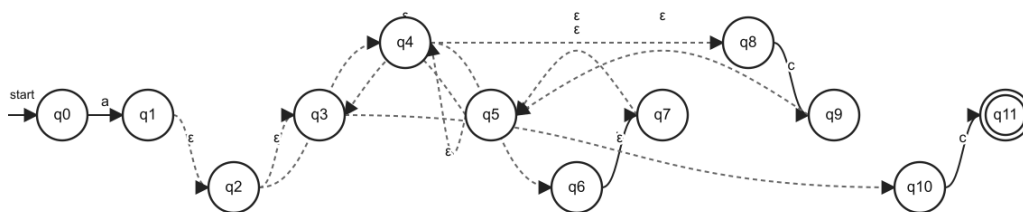
1. 根据活跃集合构造冲突图边。
2. 判断 3 个物理寄存器是否足够。
3. 如果只有 2 个物理寄存器，说明为什么需要 spill。

B 按考试标准重写答案

一、词法分析答案

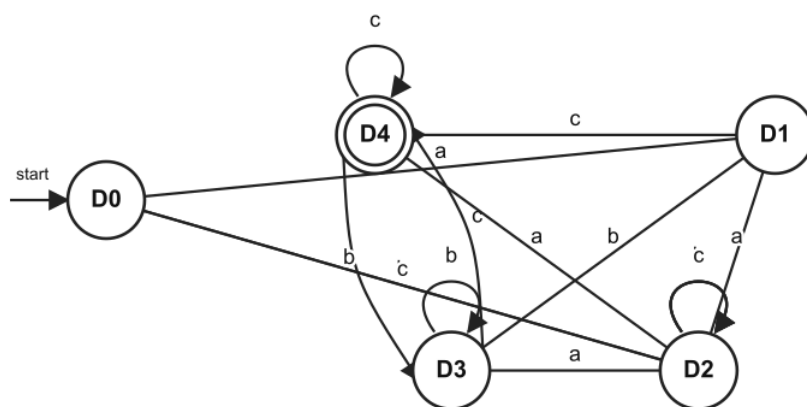
本题图形化答案如下，三张图分别对应 Thompson NFA、子集构造 DFA 和最小 DFA。

Thompson NFA: $a(b|c)^*c$



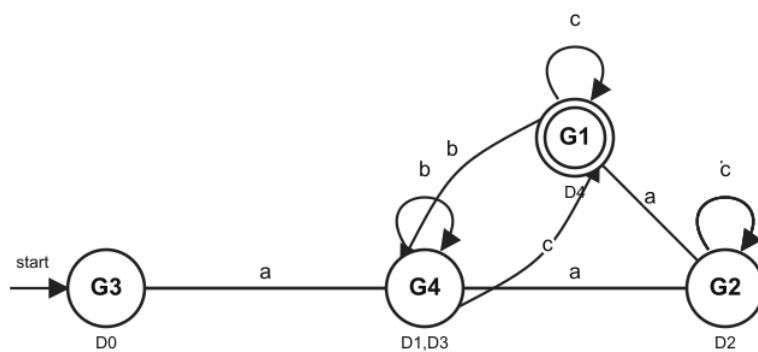
模拟卷04 Thompson NFA

Subset DFA: $a(b|c)^*c$



模拟卷04 子集构造 DFA

Minimal DFA: $a(b|c)^*c$



模拟卷04 最小 DFA

正则表达式: $a(b|c)^*c$

Thompson NFA: 起点 q_0 , 终点 q_{11} 。按下列转移表画图, q_{11} 画双圈。

$q_0 \xrightarrow{a} q_1$
 $q_6 \xrightarrow{b} q_7$
 $q_8 \xrightarrow{c} q_9$
 $q_4 \xrightarrow{\epsilon} q_6$
 $q_4 \xrightarrow{\epsilon} q_8$
 $q_7 \xrightarrow{\epsilon} q_5$
 $q_9 \xrightarrow{\epsilon} q_5$
 $q_2 \xrightarrow{\epsilon} q_4$
 $q_2 \xrightarrow{\epsilon} q_3$
 $q_5 \xrightarrow{\epsilon} q_4$
 $q_5 \xrightarrow{\epsilon} q_3$
 $q_1 \xrightarrow{\epsilon} q_2$
 $q_{10} \xrightarrow{c} q_{11}$
 $q_3 \xrightarrow{\epsilon} q_{10}$

子集构造 DFA:

D0:

NFA集合 = $\{q_0\}$

on a $\rightarrow$ D1

on b $\rightarrow$ D2

on c $\rightarrow$ D2

终态 = 否

D1:

NFA集合 = $\{q_1, q_2, q_3, q_4, q_6, q_8, q_{10}\}$

on a $\rightarrow$ D2

on b $\rightarrow$ D3

on c $\rightarrow$ D4

终态 = 否

D2:

NFA集合 = EMPTY

on a $\rightarrow$ D2

on b $\rightarrow$ D2

on c $\rightarrow$ D2

终态 = 否

D3:

NFA集合 = $\{q_3, q_4, q_5, q_6, q_7, q_8, q_{10}\}$

on a $\rightarrow$ D2

on b $\rightarrow$ D3

on c $\rightarrow$ D4

终态 = 否

D4:

NFA集合 = $\{q_3, q_4, q_5, q_6, q_8, q_9, q_{10}, q_{11}\}$

on a $\rightarrow$ D2

on b $\rightarrow$ D3

on c $\rightarrow$ D4

终态 = 是

最小化:

初分组:

终态组 = $\{D_4\}$

非终态组 = $\{D_0, D_1, D_2, D_3\}$

最终分组:

G1 = $\{D_4\}$

G2 = $\{D_2\}$

G3 = $\{D_0\}$

G4 = $\{D_1, D_3\}$

每个最终分组对应一个最小 DFA 状态。

最小化推理过程:

当前分组	检查状态	输入 a 的目标组	输入 b 的目标组	输入 c 的目标组	结论
$\{D_4\}$	D4	$\{D_0, D_1, D_2, D_3\}$	$\{D_0, D_1, D_2, D_3\}$	$\{D_4\}$	终态组单独保留
$\{D_0, D_1, D_2, D_3\}$	D1	$\{D_0, D_1, D_2, D_3\}$	$\{D_0, D_1, D_2, D_3\}$	$\{D_4\}$	与 D3 相同
$\{D_0, D_1, D_2, D_3\}$	D3	$\{D_0, D_1, D_2, D_3\}$	$\{D_0, D_1, D_2, D_3\}$	$\{D_4\}$	D1、D3 可合并
$\{D_0, D_1, D_2, D_3\}$	D0	$\{D_0, D_1, D_2, D_3\}$	$\{D_0, D_1, D_2, D_3\}$	$\{D_0, D_1, D_2, D_3\}$	输入 c 不到终态组, 与 D1、D3 不同
$\{D_0, D_2\}$	D0	$\{D_1, D_3\}$	$\{D_2\}$	$\{D_2\}$	与 D2 在输入 a 上不同
$\{D_0, D_2\}$	D2	$\{D_2\}$	$\{D_2\}$	$\{D_2\}$	D2 是死状态

因此最终最小 DFA 为：

最小 DFA 状态	原 DFA 状态集合	a	b	c	是否终态
G3	{D0}	G4	G2	G2	否
G4	{D1,D3}	G2	G4	G1	否
G2	{D2}	G2	G2	G2	否
G1	{D4}	G2	G4	G1	是

其中 G3 是初态，G1 是终态，G2 是死状态。G4 表示已经读入开头的 **a**，并且当前处在 **(b|c)*** 中但当前串末尾还不能作为最终接受；G1 表示已经满足以 **c** 结尾。

二、LL 语法分析答案

原文法：

M -> N O

N -> N comma | name

O -> semi | end

消除左递归：

M -> N O

N -> name N'

N' -> comma N' | epsilon

O -> semi | end

FIRST：

FIRST(M)={ name }

FIRST(N)={ name }

FIRST(N')={ comma, epsilon }

FIRST(O)={ semi, end }

FOLLOW：

FOLLOW(M)={ \$ }

FOLLOW(N)={ semi, end }

FOLLOW(N')={ semi, end }

FOLLOW(O)={ \$ }

预测表：

M[M,name] = M -> N O

M[N,name] = N -> name N'

M[N',comma] = N' -> comma N'

M[N',semi] = N' -> epsilon

M[N',end] = N' -> epsilon

M[O,semi] = O -> semi

M[O,end] = O -> end

结论：表中无多重入口，消除左递归后的文法是 LL(1)。

左递归消除推导：

N -> N comma | name

套用直接左递归消除规则：

A -> A alpha | beta

改写为：

A -> beta A'

A' -> alpha A' | epsilon

所以：

alpha = comma

beta = name

N -> name N'

N' -> comma N' | epsilon

FIRST 推导：

符号	推导依据	FIRST	
N	N -> name N', 右部第一个符号是终结符 name	{ name }	
N'	`N' -> comma N'	epsilon`	{ comma, epsilon }
O	`O -> semi	end`	{ semi, end }
M	M -> N O , 且 N 不能推出 epsilon	{ name }	

FOLLOW 推导:

FOLLOW 集	推导依据	结果
FOLLOW(M)	M 是开始符号	{ \$ }
FOLLOW(N)	M -> N O , N 后面是 O, 所以加入 FIRST(O)	{ semi, end }
FOLLOW(N')	N -> name N' , N' 在产生式末尾, 所以加入 FOLLOW(N)	{ semi, end }
FOLLOW(O)	M -> N O , O 在产生式末尾, 所以加入 FOLLOW(M)	{ \$ }

预测分析表填表推导:

产生式	FIRST 或 FOLLOW 依据	填入表项
M -> N O	FIRST(N O)=FIRST(N)={name}	M[M,name] = M -> N O
N -> name N'	FIRST(name N')={name}	M[N,name] = N -> name N'
N' -> comma N'	FIRST(comma N')={comma}	M[N',comma] = N' -> comma N'
N' -> epsilon	FOLLOW(N')={semi,end}	M[N',semi] = N' -> epsilon, M[N',end] = N' -> epsilon
O -> semi	FIRST(semi)={semi}	M[O,semi] = O -> semi
O -> end	FIRST(end)={end}	M[O,end] = O -> end

预测分析表 (表格形式):

非终结符	输入符号	产生式
M	name	M -> N O
N	name	N -> name N'
N'	comma	N' -> comma N'
N'	semi	N' -> epsilon
N'	end	N' -> epsilon
O	semi	O -> semi
O	end	O -> end

完整预测分析表, 空白项为 error:

非终结符	name	comma	semi	end	\$
M	M -> N O	error	error	error	error
N	N -> name N'	error	error	error	error
N'	error	N' -> comma N'	N' -> epsilon	N' -> epsilon	error
O	error	error	O -> semi	O -> end	error

每个表项最多只有一条产生式, 因此消除左递归后的文法是 LL(1)。

三、指令调度答案

定义:

指令调度在不改变程序语义的前提下重排指令，减少流水线停顿，提高并行执行效率。
 RAW：先写后读，必须保持顺序。
 WAR：先读后写，写者不能提前。
 WAW：先写后写，顺序影响最终值。

指令：

I1: $R1 = R2 + R3$
 I2: $R4 = R1 + R5$
 I3: $R2 = R6 + R7$
 I4: $R1 = R8 + R9$
 I5: $R10 = R4 + R1$

依赖：

I1 -> I2 RAW on R1
 I1 -> I3 WAR on R2
 I1 -> I4 WAW on R1
 I2 -> I4 WAR on R1
 I2 -> I5 RAW on R4
 I4 -> I5 RAW on R1

寄存器重命名：

I3: $R2_{new} = R6 + R7$
 I4: $R1_{new} = R8 + R9$
 I5 使用 $R1_{new}$

重命名后保留 RAW，WAR/WAW 被消除。一个合法调度序列：
 I1, I3, I4, I2, I5

先列每条指令的读写集合：

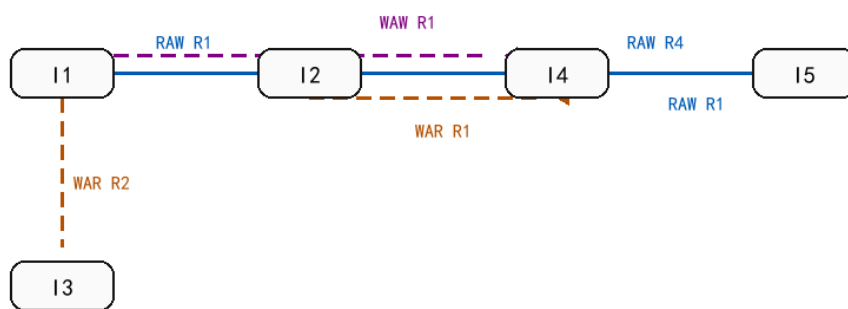
指令	读寄存器	写寄存器
I1	R2, R3	R1
I2	R1, R5	R4
I3	R6, R7	R2
I4	R8, R9	R1
I5	R4, R1	R10

逐对检查依赖：

依赖边	类型	寄存器	判断逻辑
I1 -> I2	RAW	R1	I1 写 R1, I2 读 R1, I2 必须读到 I1 的结果
I1 -> I3	WAR	R2	I1 读 R2, I3 后写 R2, I3 不能提前破坏 I1 的读值
I1 -> I4	WAW	R1	I1 写 R1, I4 后写 R1, 两个写入顺序影响最终版本
I2 -> I4	WAR	R1	I2 读 R1, I4 后写 R1, I4 不能提前破坏 I2 的读值
I2 -> I5	RAW	R4	I2 写 R4, I5 读 R4
I4 -> I5	RAW	R1	I4 写 R1, I5 读 R1, I5 需要 I4 的结果

依赖图如下。实线是 RAW，虚线是 WAR/WAW：

模拟卷04 指令依赖图：实线 RAW，虚线 WAR/WAW



寄存器重命名后，WAR/WAW 虚线约束可消除；RAW 实线约束仍必须保留。

模拟卷04 指令依赖图

寄存器重命名后：

原指令	重命名后
I3: $R2 = R6 + R7$	I3: $R2_new = R6 + R7$
I4: $R1 = R8 + R9$	I4: $R1_new = R8 + R9$
I5: $R10 = R4 + R1$	I5: $R10 = R4 + R1_new$

重命名消除了 WAR/WAW，仍必须满足：

```

I1 -> I2
I2 -> I5
I4 -> I5
    
```

序列 I1, I3, I4, I2, I5 的合法性检查：

约束	在该序列中的相对顺序	是否满足
I1 -> I2	I1 在 I2 前	是
I2 -> I5	I2 在 I5 前	是
I4 -> I5	I4 在 I5 前	是

所以 I1, I3, I4, I2, I5 是寄存器重命名后的一个合法调度序列。

四、支配问题答案

定义：

Dom(n)：入口到 n 的每条路径都经过的节点集合。
 PostDom(n)：n 到出口的每条路径都经过的节点集合。
 Dom Frontier(n)：n 支配某个前驱，但不严格支配该节点的位置。

CFG 边：
 Entry->H
 H->Body
 Body->If
 If->Body
 If->Exit

Dom 集：
 Dom(Entry)={Entry}
 Dom(H)={Entry,H}
 Dom(Body)={Entry,H,Body}
 Dom(If)={Entry,H,Body,If}

$Dom(Exit) = \{Entry, H, Body, If, Exit\}$

直接支配者:

$idom(H) = Entry$

$idom(Body) = H$

$idom(If) = Body$

$idom(Exit) = If$

支配边界:

$DF(Body) = \{Body\}$

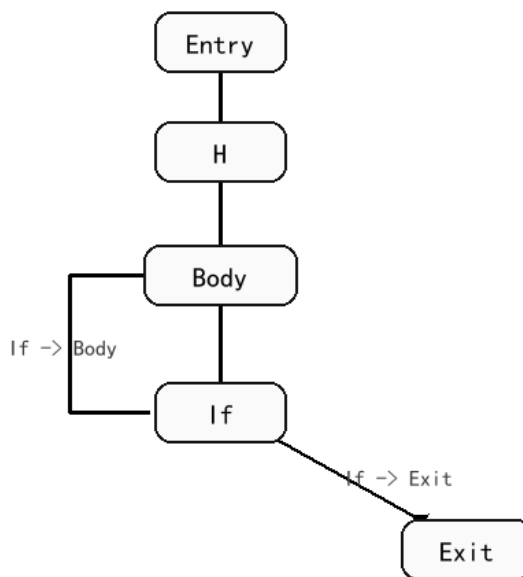
$DF(If) = \{Body\}$

其余节点 DF 为空集

SSA phi 放置: 若变量在多个基本块中被定义, 则在这些定义块的迭代支配边界处插入该变量的 phi。直观上, phi 放在多条控制流汇合且不同定义可能到达的位置

CFG 图如下:

模拟卷04 循环带出口 CFG



模拟卷04 循环带出口 CFG

Dom 集按方程计算:

$Dom(Entry) = \{Entry\}$

$Dom(n) = \{n\} \cup \text{intersection}(Dom(p))$, p 是 n 的所有前驱

逐点代入:

节点	前驱	计算过程	Dom 集
Entry	无	入口节点固定	$\{Entry\}$
H	Entry	$\{H\} \cup Dom(Entry)$	$\{Entry, H\}$
Body	H, If	$\{Body\} \cup (Dom(H) \cap Dom(If))$, 不动点结果	$\{Entry, H, Body\}$
If	Body	$\{If\} \cup Dom(Body)$	$\{Entry, H, Body, If\}$
Exit	If	$\{Exit\} \cup Dom(If)$	$\{Entry, H, Body, If, Exit\}$

直接支配者 idom:

节点	严格支配者	idom
H	$\{Entry\}$	Entry

节点	严格支配者	idom
Body	{Entry,H}	H
If	{Entry,H,Body}	Body
Exit	{Entry,H,Body,If}	If

支配边界按定义判断：若节点 X 支配 Y 的某个前驱，但 X 不严格支配 Y，则 Y 属于 DF(X)。

X	检查点	判断	DF(X)
Body	Body	Body 支配 Body 的前驱 If，且 Body 不严格支配自身	{Body}
If	Body	If 支配 Body 的前驱 If，且 If 不严格支配 Body	{Body}
Entry, H	Body	它们严格支配 Body	{}
Exit	无后继汇合	不产生支配边界	{}

因此循环变量若在 Body 或 If 中被定义，并沿回边回到 Body，则 phi 放在循环头 Body。

五、符号执行与 SAT 答案

公式：F = ((a4 and b4) or c4)
引入：x41 <-> (a4 and b4), x42 <-> (x41 or c4)，根变量 x42 为真。

CNF:
(not x41 or a4)
(not x41 or b4)
(x41 or not a4 or not b4)
(x42 or not x41)
(x42 or not c4)
(not x42 or x41 or c4)
(x42)

敏感性定义：
Path-sensitive 区分不同路径。
Flow-sensitive 区分程序点和语句顺序。
Context-sensitive 区分函数调用上下文。

同时可满足证明套用：
F1=(a0 or ... or an or c) and (b0 or ... or bm or not c), F2=(a0 or ... or an or b0 or ... or bm)。
F1 为真时，c=false 推出某个 ai=true，c=true 推出某个 bj=true，所以 F2 为真。
F2 为真时，若某个 ai=true 令 c=false；若某个 bj=true 令 c=true，所以 F1 为真。

Tseitin 转换的推导表：

子公式	新变量	等价关系	CNF 子句
a4 and b4	x41	x41 <-> (a4 and b4)	(not x41 or a4); (not x41 or b4); (x41 or not a4 or not b4)
x41 or c4	x42	x42 <-> (x41 or c4)	(x42 or not x41); (x42 or not c4); (not x42 or x41 or c4)
整个公式为真	x42	根变量取真	(x42)

所以最终 CNF 是：

(not x41 or a4)
(not x41 or b4)
(x41 or not a4 or not b4)
(x42 or not x41)
(x42 or not c4)
(not x42 or x41 or c4)
(x42)

两组公式同时可满足的证明按两个方向写：

已知	分情况	构造或推出	结论
$F1 = (a0 \text{ or } \dots \text{ or an or c}) \text{ and } (b0 \text{ or } \dots \text{ or bm or not c})$ 为真	$c = \text{false}$	第一项要为真，必须有某个 $ai = \text{true}$	$F2$ 为真
$F1$ 为真	$c = \text{true}$	第二项要为真，必须有某个 $bj = \text{true}$	$F2$ 为真
$F2 = (a0 \text{ or } \dots \text{ or an or b0 or } \dots \text{ or bm})$ 为真	存在某个 $ai = \text{true}$	取 $c = \text{false}$	$F1$ 为真
$F2$ 为真	存在某个 $bj = \text{true}$	取 $c = \text{true}$	$F1$ 为真

因此两组公式存在满足赋值的条件一致，可同时满足。

六、Andersen 指针分析答案

```

程序：
a = &O1
b = &O2
c = a
d = b
*c = d
e = *a

约束：
O1 in pts(a)
O2 in pts(b)
pts(a) subset pts(c)
pts(b) subset pts(d)
for v in pts(c), pts(d) subset pts(v)
for v in pts(a), pts(v) subset pts(e)

闭包：
pts(a)={O1}
pts(b)={O2}
pts(c)={O1}
pts(d)={O2}
由 *c=d 得 pts(O1)={O2}
由 e=*a 得 pts(e)={O2}

```

逐条语句生成约束：

语句	Andersen 约束	含义
$a = \&O1$	$O1 \text{ in pts}(a)$	a 指向 O1
$b = \&O2$	$O2 \text{ in pts}(b)$	b 指向 O2
$c = a$	$\text{pts}(a) \text{ subset pts}(c)$	a 的指向集合流入 c
$d = b$	$\text{pts}(b) \text{ subset pts}(d)$	b 的指向集合流入 d
$*c = d$	对每个 $v \text{ in pts}(c)$, $\text{pts}(d) \text{ subset pts}(v)$	d 指向的对象流入 c 所指对象
$e = *a$	对每个 $v \text{ in pts}(a)$, $\text{pts}(v) \text{ subset pts}(e)$	a 所指对象的内容流入 e

闭包计算过程：

轮次	新增事实	原因
0	$\text{pts}(a) = \{O1\}$, $\text{pts}(b) = \{O2\}$	两条取地址语句
1	$\text{pts}(c) = \{O1\}$	$c = a$
2	$\text{pts}(d) = \{O2\}$	$d = b$
3	$\text{pts}(O1)$ 加入 $O2$	$*c = d$, 且 $\text{pts}(c) = \{O1\}$ 、 $\text{pts}(d) = \{O2\}$
4	$\text{pts}(e)$ 加入 $O2$	$e = *a$, 且 $\text{pts}(a) = \{O1\}$ 、 $\text{pts}(O1) = \{O2\}$

最终 points-to 集合：

符号	points-to 集合
a	$\{O1\}$

符号	points-to 集合
b	{O2}
c	{O1}
d	{O2}
e	{O2}
O1	{O2}
O2	{}

七、PPT 新增重点答案

冲突图：
顶点是变量。
同一程序点同时活跃的两个变量之间有边。

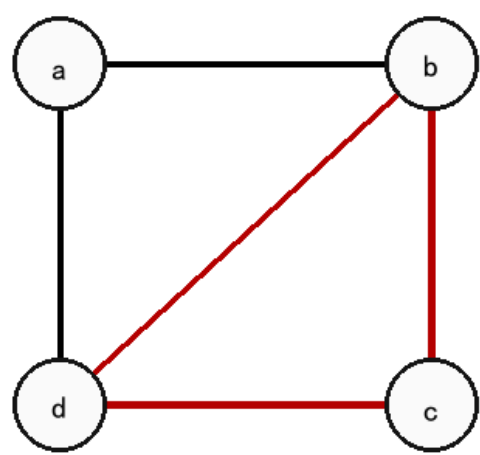
边：
(a,d)
(a,b)
(b,d)
(b,c)
(c,d)

3 个物理寄存器足够，例如：
d -> R1
b -> R2
a -> R3
c -> R3
a 和 c 没有同时活跃，可以复用 R3。

只有 2 个物理寄存器时不够：
b、c、d 两两冲突，形成三角形，需要 3 种颜色。
因此 2 个寄存器下至少需要 spill 其中一个变量。

冲突图如下：

模拟卷04 寄存器冲突图



红色三角形 b-c-d 表示三者两两冲突，因此 2 个寄存器不够。

根据活跃集合逐点构造边：

程序点	活跃集合	新增冲突边
p1	{a,d}	(a,d)
p2	{a,b,d}	(a,b), (a,d), (b,d)
p3	{b,d}	(b,d)
p4	{b,c,d}	(b,c), (b,d), (c,d)
p5	{c,d}	(c,d)

去重后的冲突边为：

(a,d), (a,b), (b,d), (b,c), (c,d)

3 个寄存器的着色表：

变量	分配寄存器	检查
d	R1	d 与 a、b、c 都冲突，所以单独用 R1
b	R2	b 与 a、c、d 冲突，不能用 R1，也不能与 a/c 同色
a	R3	a 与 b、d 冲突，不与 c 冲突
c	R3	c 与 b、d 冲突，不与 a 冲突，可和 a 复用 R3

只有 2 个寄存器时不够，因为 **b**、**c**、**d** 三个变量两两冲突，构成三角形，三角形需要 3 种颜色。因此 2 个寄存器下至少需要 spill **b**、**c**、**d** 中的一个变量。